

Task- ASG

1. Create one VPC in N. Virginia region.

Objective

Create a Virtual Private Cloud in the N. Virginia AWS region

Purpose

A VPC provides an isolated networking environment in AWS

Step 1 – Select Region

Step 2 – Open VP

Step 3 – Create VPC

VPC

→ Your VPCs

→ Create VPC

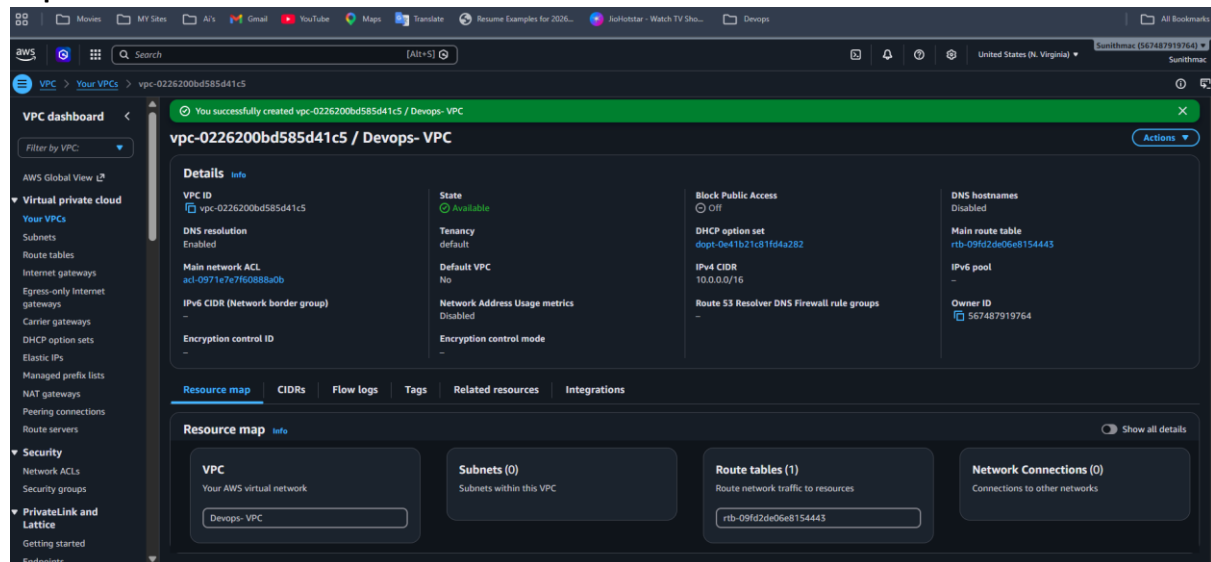
Name: DevOps-VPC

CIDR: 10.0.0.0/16

State: Available

Region: us-east-1

Implementation:



Conclusion

The VPC has been successfully created in the N. Virginia region. This VPC will serve as the main networking environment for the entire project

2. Create two subnets: one public subnet and one private subnet.

Objective

Create separate public and private subnets inside the VPC

Purpose

Subnets allow us to divide the VPC into different network segments.

Task- ASG

VPC

10.0.0.0/16

Public Subnet

10.0.1.0/24

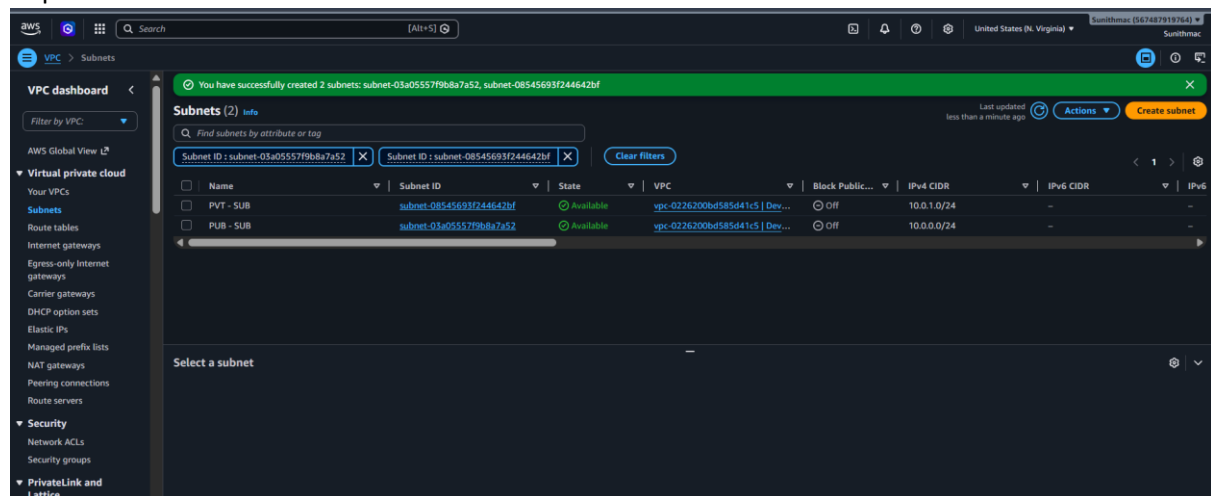
Private Subnet

10.0.2.0/24

Public Subnet 2

10.0.3.0/24

Implementation:



Conclusion

The VPC has been divided into public and private network segments. The additional public subnet provides a second Availability Zone for the Application Load Balancer.

3. Attach an IGW to the VPC.

Objective

Create an Internet Gateway and attach it to the VPC.

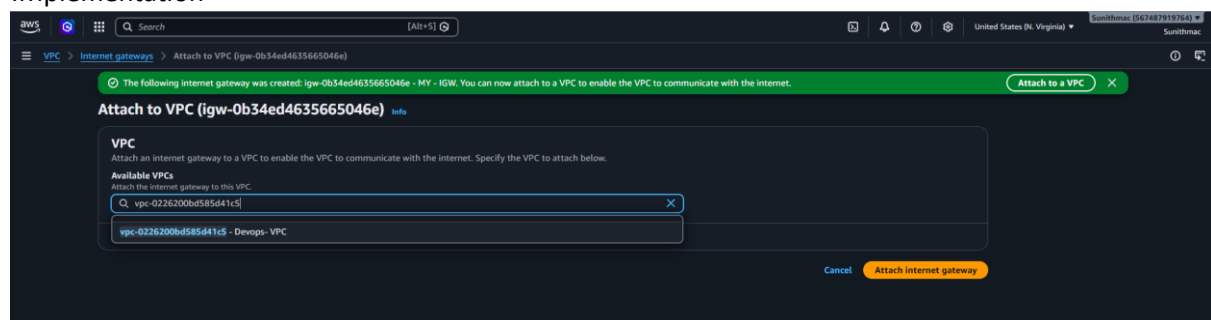
Purpose

The Internet Gateway provides Internet connectivity between the VPC and the Internet.

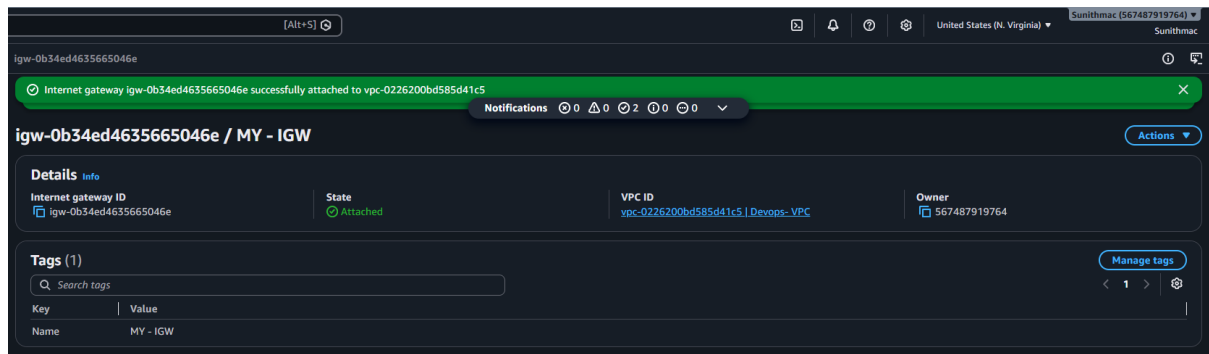
Public EC2 → Public Route Table → Internet Gateway → Internet

Internet Gateway+ Route Table + Public Route + Public IP Address

Implementation



Task- ASG



Conclusion

The Internet Gateway has been successfully attached to the VPC and is ready to provide Internet connectivity to public resources.

4. Create one public route table (RT) and one private route table.

Objective

Create a route table for the public subnet.

Purpose

The public route table directs Internet-bound traffic from the public subnet to the Internet Gateway.

Steps

Open **VPC → Route Tables**.

Click **Create route table**.

Enter:

Name: DevOps-Public-RT

VPC: DevOps-VPC

Click **Create route table**.

Select DevOps-Public-RT.

Open **Routes → Edit routes**.

Add:

Destination: 0.0.0.0/0

Target: DevOps-IGW

Save the route.

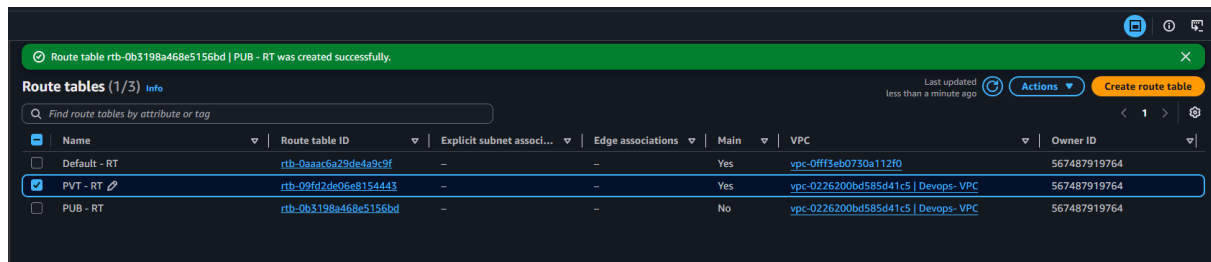
Open **Subnet associations**.

Click **Edit subnet associations**.

Select DevOps-Public-Subnet.

Save.

Task- ASG



Conclusion

The public subnet is now configured to send Internet-bound traffic through the Internet Gateway.

5. Deploy a NAT gateway in the public subnet and attach the NAT gateway to the private subnet.

Objective

Create a NAT Gateway in the public subnet.

Purpose

The NAT Gateway allows private EC2 instances to initiate outbound Internet connections without requiring public IP addresses.

Steps — Allocate Elastic IP

Open **VPC Console**.

Select **Elastic IPs**.

Click **Allocate Elastic IP address**.

Select **Amazon's pool of IPv4 addresses**.

Click **Allocate**.

Steps — Create NAT Gateway

Open **VPC → NAT Gateways**.

Click **Create NAT gateway**.

Enter:

Name: DevOps-NAT-Gateway

Subnet: DevOps-Public-Subnet

Connectivity type: Public

Elastic IP: Select the allocated Elastic IP

Click **Create NAT gateway**.

Wait until the state changes to **Available**.

Configure Private Route

Open **VPC → Route Tables**.

Select DevOps-Private-RT.

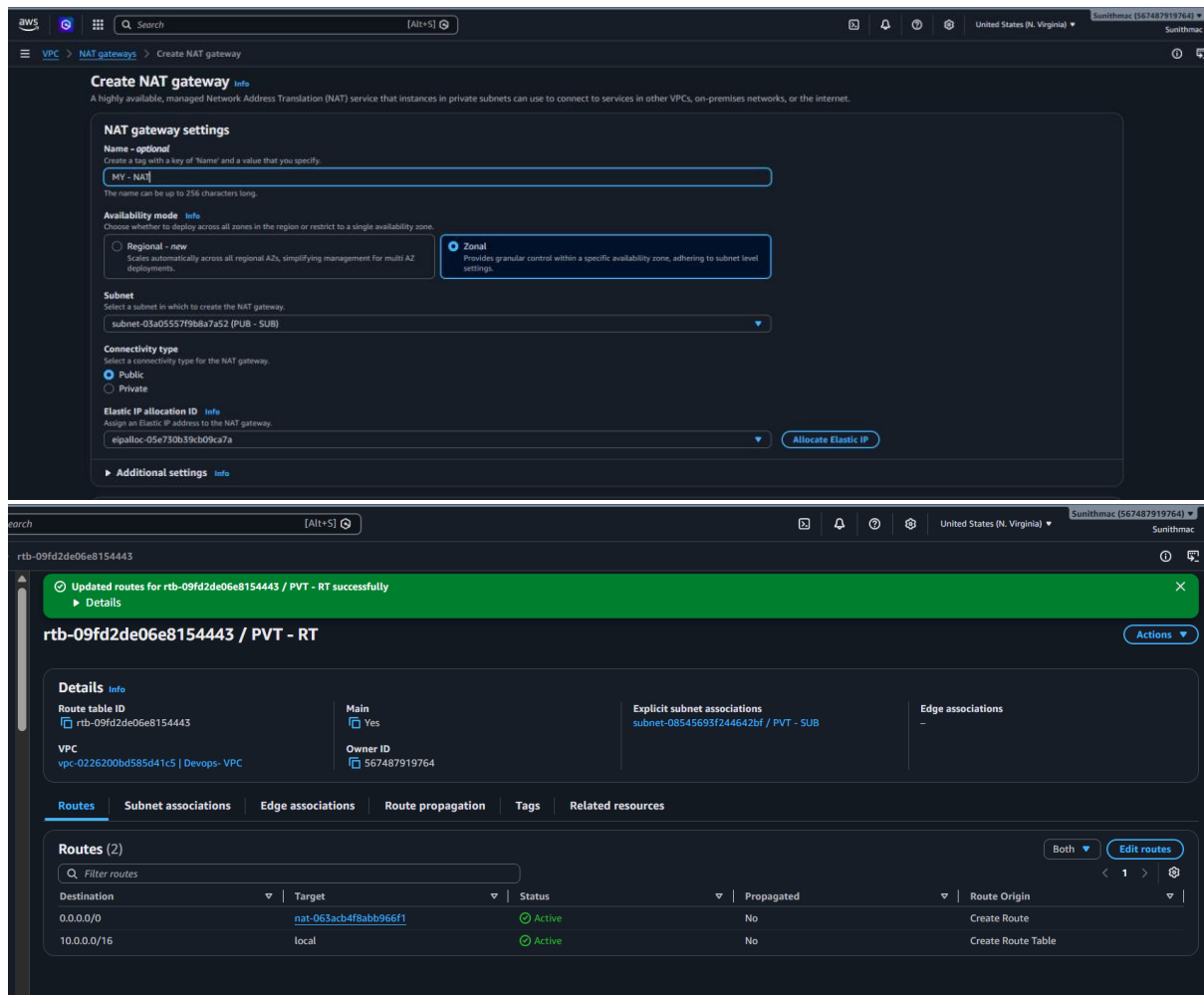
Open **Routes → Edit routes**.

Add:

Destination: 0.0.0.0/0

Target: DevOps-NAT-Gateway

Task- ASG



Conclusion

The private subnet can now access the Internet for outbound connections through the NAT Gateway while remaining private.

Note: NAT Gateway is a chargeable AWS resource. Delete it after completing the lab.

6. Create two instances, one in the public subnet and one in the private subnet.

Objective

Launch an EC2 instance in the public subnet.

Purpose

The public EC2 instance will host Apache and demonstrate a server with public Internet connectivity.

Steps

Open **EC2 Console**.

Select **Instances**.

Click **Launch instance**.

Enter:

Name: DevOps-Public-EC2

Select an Amazon Linux AMI.

Select an instance type eligible for your AWS account.

Select or create an EC2 key pair.

Under Network settings, click **Edit**.

Task- ASG

Select:

VPC: DevOps-VPC

Subnet: DevOps-Public-Subnet {select PVT for Private Instance}

Auto-assign Public IP: Enable

Create/select a security group.

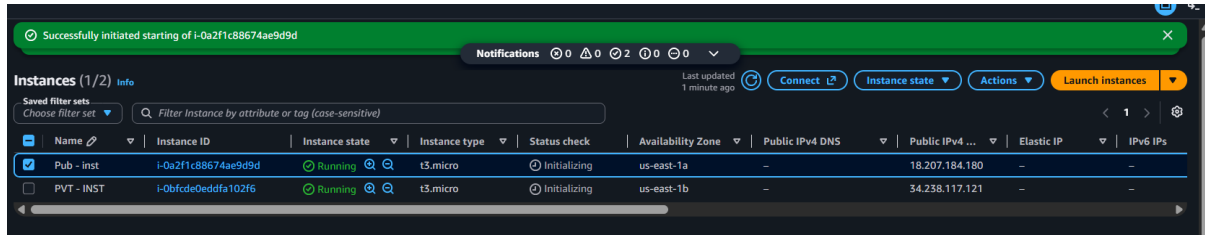
Security Group

Allow:

SSH → Port 22 → My IP

HTTP → Port 80 → 0.0.0.0/0

Launch the instance.



Conclusion

The public and private EC2 instance was successfully launched and is ready to host the Apache web server.

7. Deploy Apache server on both EC2 instances with a sample index.html file.

Objective

Install and configure Apache HTTP Server on the EC2 instances.

Purpose

Apache will act as the web server and provide the application content that users access through the Application Load Balancer

Steps — Public EC2

Connect to the public EC2 instance.

```
ssh -i DevOps-Key.pem ec2-user@PUBLIC-IP
```

Install Apache:

```
sudo dnf install httpd -y
```

Start Apache:

```
sudo systemctl start httpd
```

Enable Apache:

```
sudo systemctl enable httpd
```

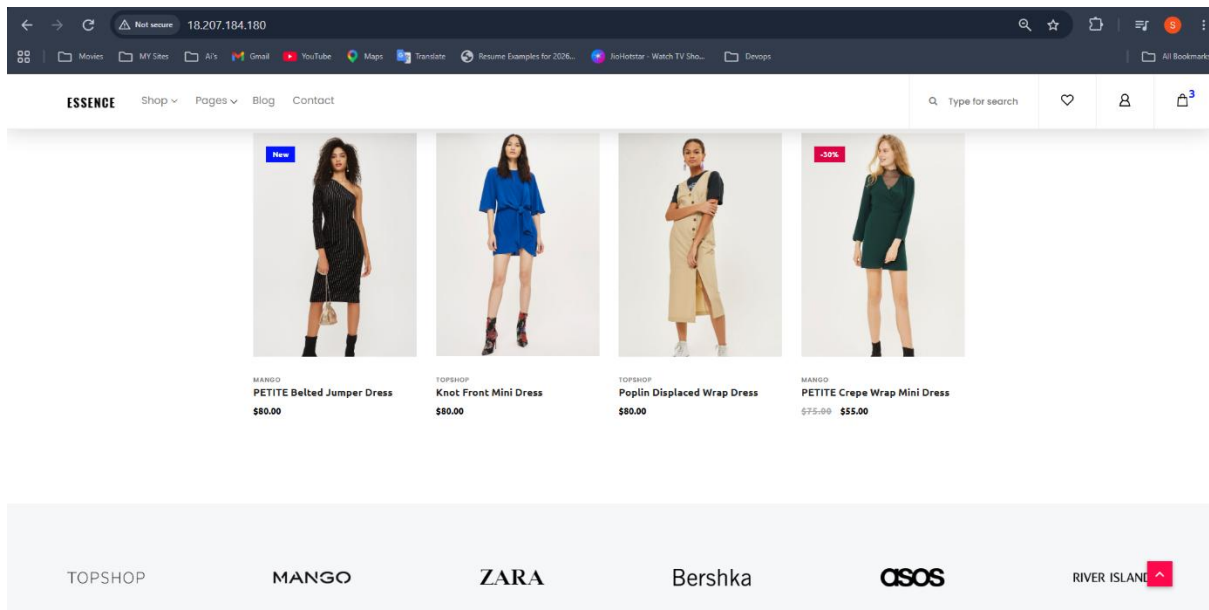
Check status:

```
sudo systemctl status httpd
```

Create a web page:

```
sudo git clone <linkpaste> /var/www/html/index.html >
```

Task- ASG



Conclusion

Apache was successfully installed on both EC2 instances, making them ready to serve web traffic.

8. Create one application load balancer and attach it to both EC2 instances.

Objective

Create an internet-facing Application Load Balancer to distribute HTTP traffic across the EC2 instances.

Purpose

The ALB provides a single endpoint for users and distributes requests across healthy application servers.

It also works with Auto Scaling to automatically add or remove application instances.

Create Target Group

Open **EC2** → **Target Groups**.

Click **Create target group**.

Target type: **Instances**.

Name:

DevOps-TG

Protocol: HTTP.

Port: 80.

VPC: DevOps-VPC.

Health check protocol: HTTP.

Health check path:

/

Register the EC2 instances.

Create ALB

Open **EC2** → **Load Balancers**.

Click **Create Load Balancer**.

Task- ASG

Select **Application Load Balancer**.

Enter:

Name: DevOps-ALB

Scheme: Internet-facing

IP address type: IPv4

Select DevOps-VPC.

Select two public subnets in different Availability Zones.

select the ALB security group.

Configure listener:

HTTP : 80

Forward traffic to:

DevOps-TG

Create the load balancer.

The image displays two screenshots of the AWS Management Console, illustrating the setup and monitoring of an Application Load Balancer (ALB).

Top Screenshot: ALB Configuration

- Target type:** Instance
- Protocol : Port:** HTTP: 80
- Protocol version:** HTTP1
- VPC:** vpc-0226200bd585d41c5
- IP address type:** IPv4
- Load balancer:** None associated
- Targets:** 2 Total targets, 0 Anomalous, 0 Healthy, 0 Unhealthy, 2 Unused, 0 Initial, 0 Draining.
- Distribution of targets by Availability Zone (AZ):** Select values in this table to see corresponding filters applied to the Registered targets table below.
- Registered targets (2):** Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.
- Targets Table:**

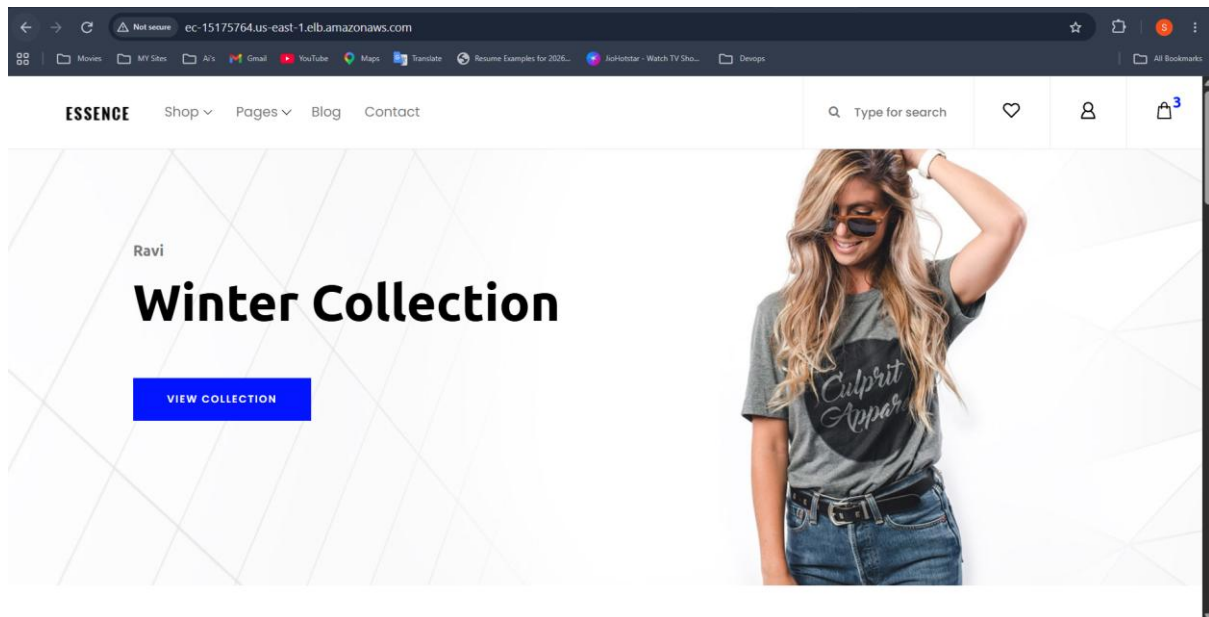
Instance ID	Port	Zone	Health status	Health status details	Administrative o...	Override detail
i-02904cd2d30ec2a42	80	us-east-1b (us...	Unused	Target group is not configured to receive traffic from the load balancer	-	-
i-0a2f1c88674ae9d9d	80	us-east-1a (us...	Unused	Target group is not configured to receive traffic from the load balancer	-	-

Bottom Screenshot: ALB Monitoring

- Details:** arn:aws:elasticloadbalancing:us-east-1:567487919764:targetgroup/app-data/e306c627390d7c43
- Target type:** Instance
- Protocol : Port:** HTTP: 80
- Protocol version:** HTTP1
- VPC:** vpc-0226200bd585d41c5
- IP address type:** IPv4
- Load balancer:** App-elb
- Targets:** 2 Total targets, 0 Anomalous, 2 Healthy, 0 Unhealthy, 0 Unused, 0 Initial, 0 Draining.
- Distribution of targets by Availability Zone (AZ):** Select values in this table to see corresponding filters applied to the Registered targets table below.
- Registered targets (2):** Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.
- Targets Table:**

Instance ID	Name	Port	Zone	Health status	Health status details	Administrative o...	Override details	Launch...	Anom...
-------------	------	------	------	---------------	-----------------------	---------------------	------------------	-----------	---------

Task- ASG



Conclusion

The Application Load Balancer was successfully created and configured to distribute HTTP requests to healthy EC2 instances.

9. Store application load balancer logs in S3.

Objective

Create an S3 bucket and configure the ALB to store access logs.

Purpose

ALB access logs provide information about requests received by the load balancer and responses from the targets.

They are useful for troubleshooting, auditing, and traffic analysis.

Steps

Create S3 Bucket

Open **Amazon S3**.

Click **Create bucket**.

Enter a globally unique bucket name, for example:

devops-alb-logs-unique-name

Select region:

us-east-1

Keep **Block Public Access** enabled.

Create the bucket.

Enable ALB Logging

Open **EC2 → Load Balancers**.

Select DevOps-ALB.

Open **Attributes**.

Find **Access logs**.

Enable access logging.

Select the S3 bucket.

Save the configuration.

Task- ASG

The first screenshot shows the 'Edit load balancer attributes' page for an Application Load Balancer. The 'Cross-zone load balancing' checkbox is checked. Under 'ARC zonal shift integration', the 'Disable - Default' radio button is selected. The 'Protection' section shows 'Deletion protection' is disabled. The 'Logging' section shows 'Access logs - legacy' is checked, and the 'S3 URI' is configured as 's3://devops-alb-logs321'. The 'Logging options' section indicates that log delivery to CloudWatch, Kinesis Firehose, and S3 is configurable through an integration with Logging - by Amazon CloudWatch.

The second screenshot shows the 'ELBAccessLogTestFile' object in the Amazon S3 console. The 'Object overview' section displays the object's details: Owner (60f256fa43758d411b1de59ae67f034235f6ca808cea2fc286a412a0ccbcacbf), AWS Region (US East (N. Virginia) us-east-1), Last modified (September 11, 2026, 17:08:01 (UTC+05:30)), Size (77.0 B), Type (Key), and Key (AWSLogs/567487919764/ELBAccessLogTestFile). The 'Object management overview' section provides information about bucket properties and management configurations.

The third screenshot shows the 'devops-alb-logs321' bucket in the Amazon S3 console. The 'Objects (1)' section displays a table with one object: 'AWSLogs/'. The table columns are Name, Type, Last modified, Size, and Storage class.

Conclusion

ALB access logging was configured successfully, allowing load balancer traffic to be stored in Amazon S3 for future analysis

Task- ASG

10.Store the VPC flow logs in a CloudWatch log group.

Objective

Enable VPC Flow Logs and send the logs to Amazon CloudWatch Logs.

Purpose

VPC Flow Logs provide visibility into network traffic and can help troubleshoot connectivity, security group rules, and rejected traffic.

Steps

Create CloudWatch Log Group

Open **CloudWatch**.

Select **Logs** → **Log groups**.

Click **Create log group**.

Enter: /aws/vpc/devops-flow-logs

Create the log group.

Create IAM Role

Open **IAM**.

Select **Roles**.

Click **Create role**.

Select the appropriate AWS service/trust configuration for VPC Flow Logs.

Grant the required CloudWatch Logs permissions.

Name the role: DevOps-VPC-FlowLogs-Role

Enable Flow Logs

Open **VPC** → **Your VPCs**.

Select DevOps-VPC.

Choose **Actions** → **Create flow log**.

Enter: Name: DevOps-VPC-FlowLogs

Filter: All

Maximum aggregation interval: 1 minute

Destination: CloudWatch Logs

Log group: /aws/vpc/devops-flow-logs

IAM role: DevOps-VPC-FlowLogs-Role

Create the flow log.

Verification

Go to:

CloudWatch

→ Logs

→ Log groups

→ /aws/vpc/devops-flow-logs

Verify that log streams and flow log records appear.

Task- ASG

The first screenshot shows the AWS CloudWatch console with a notification that the log group `/aws/vpc/devops-flow-logs` has been created. The 'Log groups' tab is active, showing a table with one log group: `/aws/vpc/devops-flow-logs` with a 'Standard' log class and 'Never expire' retention.

The second screenshot shows the VPC dashboard for 'Devops-VPC' (vpc-0226200bd585d41c5). The 'Flow logs' tab is selected, showing a single flow log named 'devops' with ID 'fl-02c6081436e92092b'. The destination is 'cloud-watch-logs' pointing to the `/aws/vpc/devops-flow-logs` log group.

The third screenshot shows the 'Log streams' tab for the `/aws/vpc/devops-flow-logs` log group. It displays five log streams with their last event times:

Log stream	Last event time
<code>eni-018ade42f986721f0-all</code>	2026-09-11 11:59:05 (UTC)
<code>eni-0cbe8f6dae86509b-all</code>	2026-09-11 11:59:00 (UTC)
<code>eni-0ac14b4af8780acda-all</code>	2026-09-11 11:58:48 (UTC)
<code>eni-02a6555def4e97da9-all</code>	2026-09-11 11:58:46 (UTC)
<code>eni-043154499ef99a429-all</code>	2026-09-11 11:58:19 (UTC)

Conclusion

VPC Flow Logs were successfully configured to send network traffic information to CloudWatch Logs.

Task- ASG

11. Create monitoring dashboards to monitor CPU utilization and to monitor the Apache service.

Objective

Create a CloudWatch dashboard to monitor EC2 CPU utilization and Apache service health.

Purpose

CloudWatch provides centralized monitoring of AWS infrastructure and applications.

Monitoring CPU utilization helps identify resource usage, while Apache monitoring helps determine whether the web service is running.

Steps

Create Dashboard

Open **CloudWatch**.

Select **Dashboards**.

Click **Create dashboard**.

Enter:

DevOps-Monitoring

Select a widget such as **Line**.

Select EC2 metrics.

Select: CPUUtilization

Add the public and private EC2 instances.

Select an appropriate period, such as 5 minutes.

Save the widget.

Apache Monitoring

Apache service status is not automatically available as a standard EC2 metric.

A custom CloudWatch metric can be created.

Example:

Namespace: DevOps/Apache

Metric: ApacheStatus

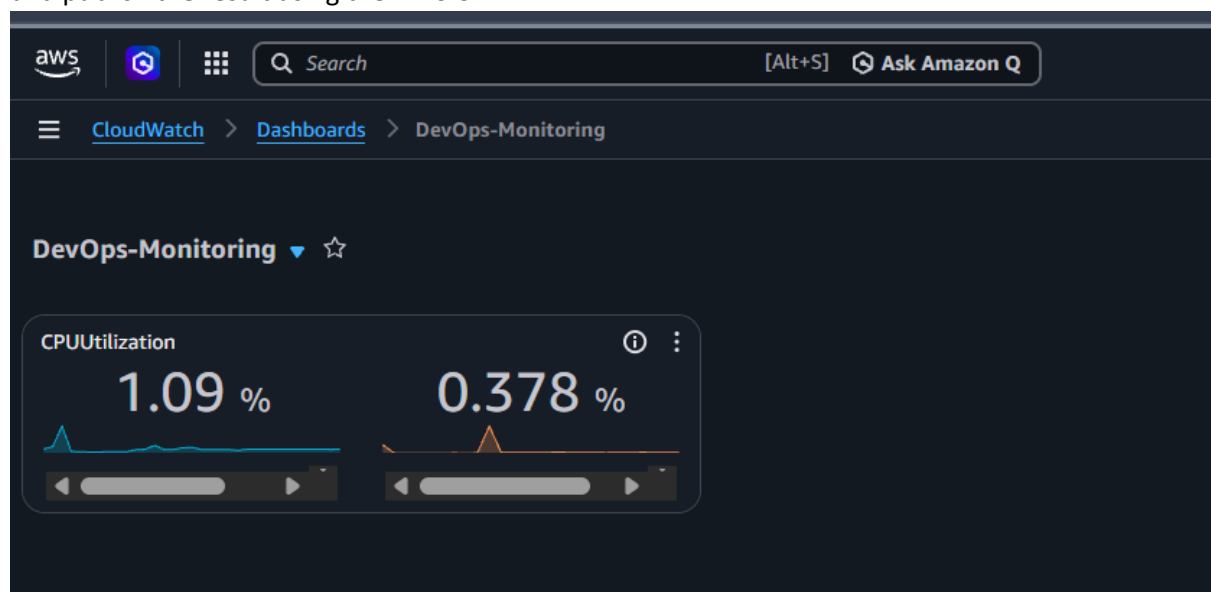
Value: 1 = Running

0 = Stopped

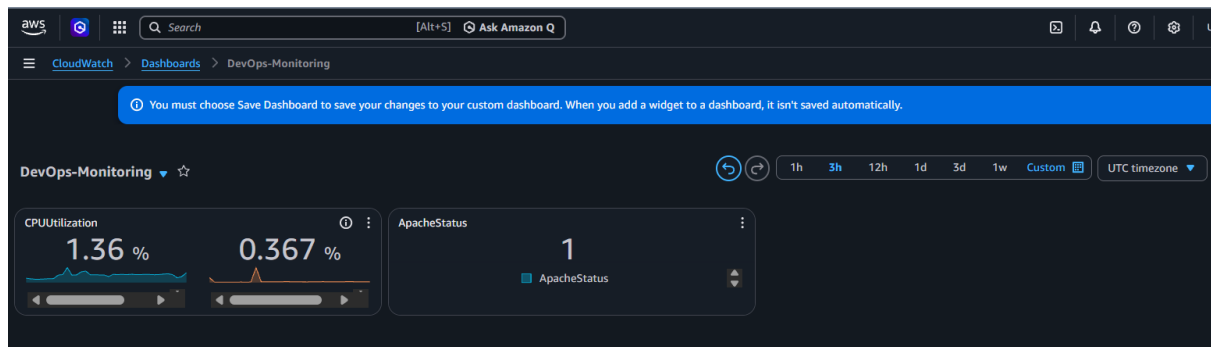
A monitoring script can check:

systemctl is-active httpd

and publish the result using the AWS CLI



Task- ASG



Conclusion

The CloudWatch dashboard provides centralized visibility into EC2 CPU performance and Apache application health

12.If CPU utilization is more than 70%, then it should trigger auto scaling and launch new instance.

Objective

Create an EC2 Launch Template that Auto Scaling can use to launch new web servers automatically.

Purpose

The Launch Template defines the configuration for new EC2 instances, including the AMI, instance type, security group, and startup commands.

Steps

Open **EC2**.

Select **Launch Templates**.

Click **Create launch template**.

Enter:

Name:DevOps-Web-Launch-Template

Select an Amazon Linux AMI.

Select an eligible instance type.

Select the required key pair.

Select the web server security group.

Under **Advanced details**, locate **User data**.

Verification

Confirm the Launch Template exists and contains the required AMI, instance type, security group, and User Data.

Create Auto Scaling Group

Steps

Open **EC2** → **Auto Scaling Groups**.

Click **Create Auto Scaling group**.

Enter:

Name:DevOps-ASG

Select:DevOps-Web-Launch-Template

Select the VPC:DevOps-VPC

Select appropriate subnets in multiple Availability Zones.

Configure load balancing.

Select **Attach to an existing load balancer**.

Select the target group:DevOps-TG

Configure capacity:

Task- ASG

Minimum: 2

Desired: 2

Maximum: 4

Enable appropriate health checks.

Select **Target tracking scaling policy**.

Choose average CPU utilization.

Set target: 70%

The screenshot displays the AWS Management Console interface, showing the configuration of an Auto Scaling Group (ASG) and the resulting EC2 instances.

Auto Scaling groups (1) info

Name	Status - new	Launch template/configuration	Instances	Instance health - new	Desired capacity	Min	Max	Availability
ec-app-ASG	Deleting	ec-ASG Version Default	2	2/2 Healthy	0	0	0	3 Availabil

Next Steps

- Launch an Instance**
With On-Demand Instances, you pay for compute capacity by the second (for Linux, with a minimum of 60 seconds) or by the hour (for all other operating systems) with no long-term commitments or upfront payments. Launch an On-Demand Instance from your launch template.
[Launch instance from this template](#)
- Create an Auto Scaling group from your template**
Amazon EC2 Auto Scaling helps you maintain application availability and allows you to scale your Amazon EC2 capacity up or down automatically according to conditions you define. You can use Auto Scaling to help ensure that you are running your desired number of Amazon EC2 instances during demand spikes to maintain performance and decrease capacity during lulls to reduce costs.
[Create Auto Scaling group](#)
- Create Spot Fleet**
A Spot Instance is an unused EC2 instance that is available for less than the On-Demand price. Because Spot Instances enable you to request unused EC2 instances at steep discounts, you can lower your Amazon EC2 costs significantly. The hourly price for a Spot Instance (of each instance type in each Availability Zone) is set by Amazon EC2, and adjusted gradually based on the long-term supply of and demand for Spot Instances. Spot instances are well-suited for data-analysis, batch jobs, background processing, and optional tasks.
[Create Spot Fleet](#)

Instances (5) info

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
diff az	i-0ea54b31b1b0667e	Running	t3.micro	3/3 checks passed	us-east-1c	-	18.212.227.0	-	-
	i-042628c205446d8e0	Running	t3.micro	Initializing	us-east-1b	-	-	-	-
	i-02de9603af08b132a	Running	t3.micro	Initializing	us-east-1c	-	-	-	-
Pub - inst	i-0a271c88674ae9d9d	Running	t3.micro	3/3 checks passed	us-east-1a	-	18.209.240.98	-	-
pvt	i-02904cd7d30ec2a42	Running	t3.micro	3/3 checks passed	us-east-1b	-	-	-	-

All Instances

- Instance alarm states**
In alarm
[View in CloudWatch](#)
- Service health**
EC2 service status
Operating normally
[View Service Health Dashboard](#)
- Health events affecting EC2 instances**
Open events
Upcoming events
[View AWS Health Dashboard](#)

Conclusion

The Auto Scaling Group was successfully configured to maintain the required number of EC2 instances and automatically scale capacity based on CPU utilization.